



LECTURE 3

Heba Sabri

AGENDA

- Pipes
- Sharing Data Between Components
- Life cycle Methods

PIPES

- Pipes are simple functions you can use in template expressions to accept an input value and return a transformed value.
- Angular provides built-in pipes for typical data transformations like :
 - DatePipe: Formats a date value according to locale rules.
 - `{{ today | date : 'MMMM YYYY' }}`
 - UpperCasePipe: Transforms text to all upper case.
 - `{{ name | uppercase }}`
 - LowerCasePipe: Transforms text to all lower case.
- Learn More : <https://angular.dev/guide/templates/pipes>

CUSTOM PIPES

- To generate custom pipe you need to run :

`ng generate pipe pipeName`

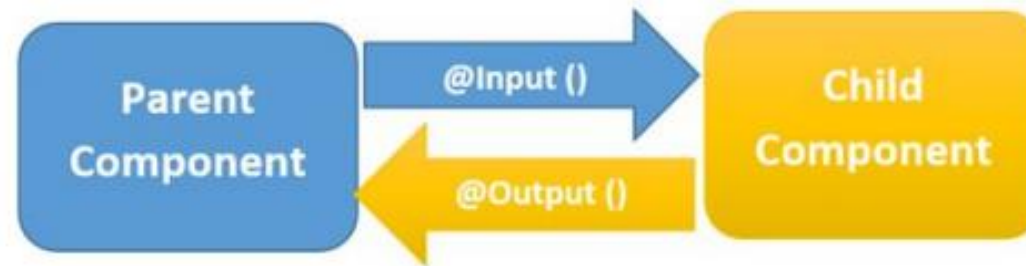
- For example you can generate custom pipe that transform file size to MB :

```
transform(value: any, ...args: any[]): unknown {  
    return (value / (1024 * 1024)).toFixed(2) + 'MB';  
}
```

SHARING DATA BETWEEN COMPONENTS

- **Parent to child :**

This is probably the most common and straightforward method of sharing data. It works by using the `@Input()` decorator to allow data to be passed via the template.



SHARING DATA BETWEEN COMPONENTS

- **Child to parent :**

Using output and event emitter is a way to share data is to emit data from the child, which can be listened to by the parent. This approach is ideal when you want to share data changes that occur on things like button clicks, form entries, and other user events.

Example :

```
<app-child (messageEvent)="receiveMessage($event)"></app-child> - parent
receiveMessage($event) {this.message = $event}- parent.ts
@Output() messageEvent = new EventEmitter<string>();- Child.ts
sendMessage() {this.messageEvent.emit(this.message)} - Child.ts
<button (click)="sendMessage()">Send Message</button> - Child.html
```

SHARING DATA BETWEEN COMPONENTS

- **Unrelated Components**
 - When passing data between components that lack a direct connection, such as siblings, grandchildren, etc, you should use a shared service.
 - And you can also create a service to set and get values across unrelated components.
- Will be covered in details later with services

COMPONENT LIFECYCLE

- A component instance has a lifecycle that starts when Angular instantiates the component class and renders the component view along with its child views.
- You don't have to implement all (or any) of the lifecycle hooks, just the ones you need.
- After your application instantiates a component or directive by calling its constructor, Angular calls the hook methods you have implemented at the appropriate point in the lifecycle of that instance.

COMPONENT LIFECYCLE

- **ngOnInit()**

Initialize the directive or component after Angular first displays the data-bound properties and sets the directive or component input properties.

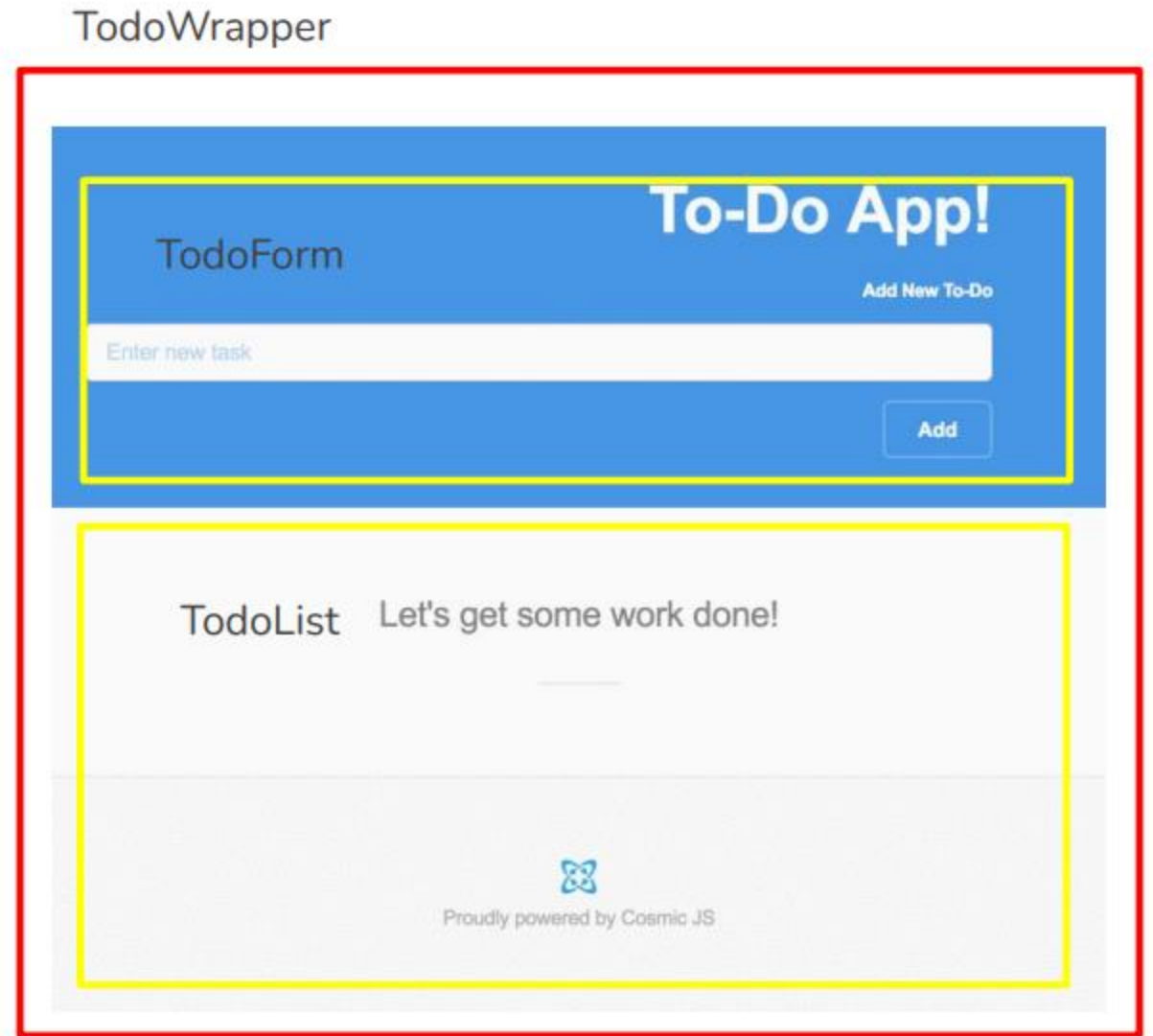
- **ngOnDestroy()**

Cleanup just before Angular destroys the directive or component. Unsubscribe Observables and detach event handlers to avoid memory leaks.

- Learn more: <https://angular.dev/guide/components/lifecycle>

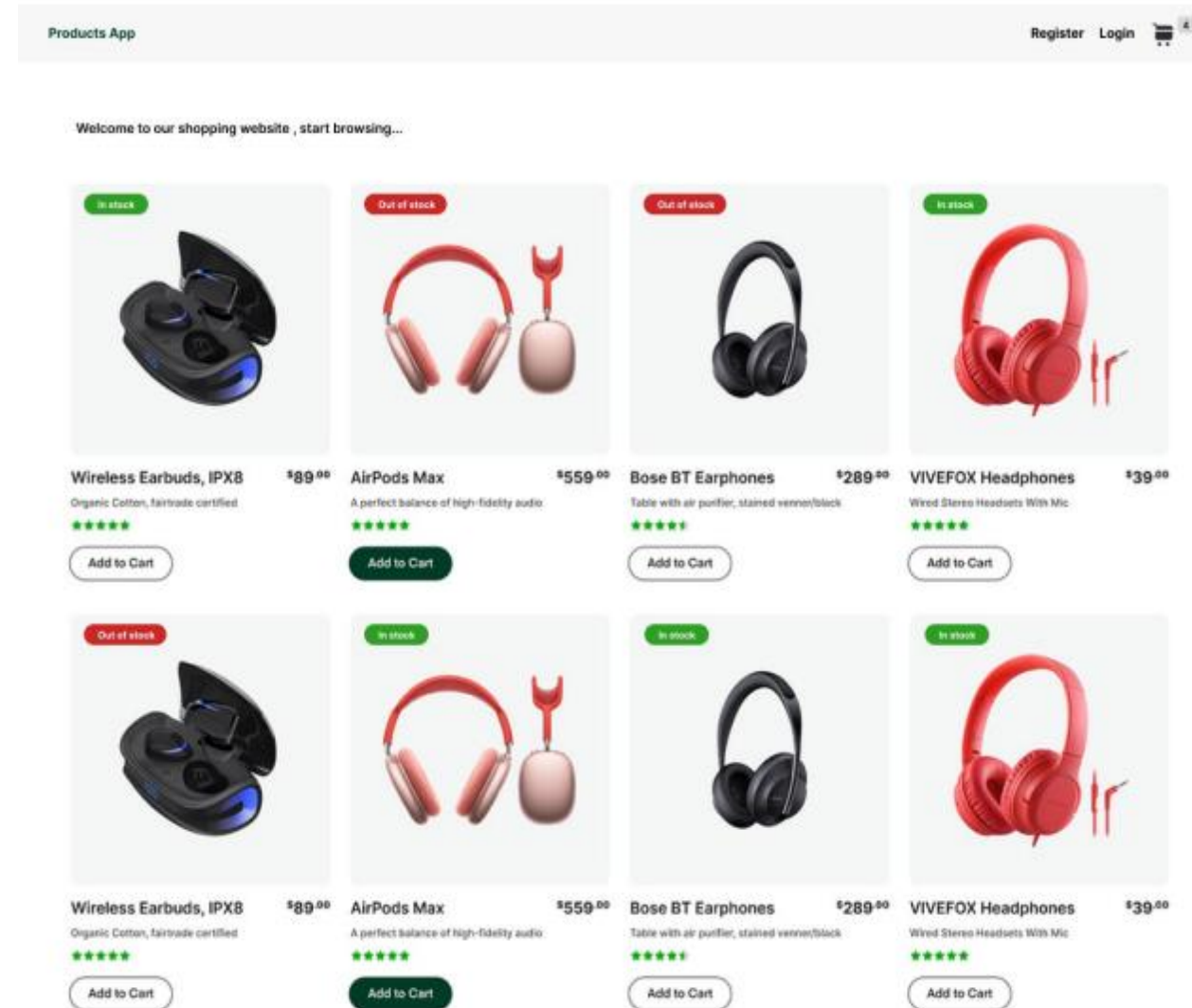
TASK

- **Red** is the parent component for the To-Do (Array of todos)
- **Yellow** borders are child components, one for the input with add button [which will emit the input value to the parent component]
- The other **yellow** border is for the other child component of the items list.
- User can add new task
- User can delete Task
- User can mark as completed



TASK

- Render Products from the provided products array [attached]
- Each product card is separate component
- If stock= 0 will return out of stock else will return in stock
- If stock= 0 => out of stock text will be red
- If stock> 0 => In stock text will be green



TASK

- Each product card item should have:
 - Product images
 - Product name
 - Product price: 20 EGP (Use pipes to render Currency)
 - Rate (Bonus if used stars to show rate)
 - “Add to cart” button
 - Use interface to define the product type

